MLOps Phase 2 Documentation
Agentic AI on Phase 1 RAG

Torob Product Retrieval Project

June 5, 2026

Abstract

This document explains the Phase 2 implementation of the MLOps project. The goal of Phase 2 is to build an Agentic AI layer on top of the Phase 1 RAG system. The agent exposes a judge-compatible `/chat` API, uses the Phase 1 product artifacts as a retrieval tool, keeps short-term conversation memory by `chat_id`, and returns product-level and seller-level random keys in the required JSON format.

Contents

1 Project Location **2**

2 Requirement-by-Requirement Implementation **2**

3 High-Level Architecture **3**

4 Main Components **3**
- 4.1 HTTP Server . . . 3
- 4.2 Agent Core . . . 3
- 4.3 Phase 1 RAG Tool . . . 4
- 4.4 Memory . . . 4
- 4.5 Monitoring . . . 5

5 API Contract **5**
- 5.1 Endpoint . . . 5
- 5.2 Request Format . . . 5
- 5.3 Response Format . . . 5

6 How Each Required Scenario Is Handled **6**
- 6.1 Scenario 1: Question About a Product Feature . . . 6
- 6.2 Scenario 2: Question About Sellers or Prices . . . 6
- 6.3 Scenario 3: Multi-Turn Seller Hunt . . . 6
- 6.4 Scenario 4: Comparing Products . . . 6
- 6.5 Scenario 5: Image to Base . . . 7

7 Guardrails **7**

8 How to Run **7**
8.1 Install Requirements 7
8.2 Start the Server 7
8.3 Browser Test Page 7
8.4 Health Check 8

9 How to Test **8**
9.1 Built-in Tests 8
9.2 Presentation Demo 8
9.3 Manual PowerShell Test 8

10 Example Outputs **8**
10.1 Seller Price Example 8

11 Limitations **9**

12 Conclusion **9**

1 Project Location

The Phase 2 implementation is placed in a fully separated folder:

```
<PROJECT_ROOT>\phase2
```

The Phase 1 source and artifacts are kept in the separated `phase1` folder. Phase 2 reads the existing Phase 1 outputs from:

```
<PROJECT_ROOT>\phase1\torob_rag\artifacts
```

2 Requirement-by-Requirement Implementation

Documentation Requirement	**Implemented Solution**	**Main Files**
Agent with tool use	The agent detects the user intent and calls the RAG tool when product data is required.	`agent.py`, `rag.py`
RAG from Phase 1 as a tool	The Phase 1 cleaned product and member artifacts are loaded and queried by a dedicated tool class.	`rag.py`
Short-term memory	The service stores recent messages and extracted slots by `chat_id`.	`memory.py`
API endpoint for judge	The project exposes `POST /chat` with the required JSON contract.	`server.py`
CLI or UI	A CLI is provided, and `GET /` serves a small browser test page.	`cli.py`, `server.py`
Monitoring	Events and metrics are written to local runtime files.	`monitoring.py`
Guardrails	Prompt-injection style requests are refused, and unknown requests are answered with uncertainty.	`agent.py`
Test scenarios	Smoke tests and example scenarios are provided.	`tests/`, `docs/test_scenarios.md`

3 High-Level Architecture

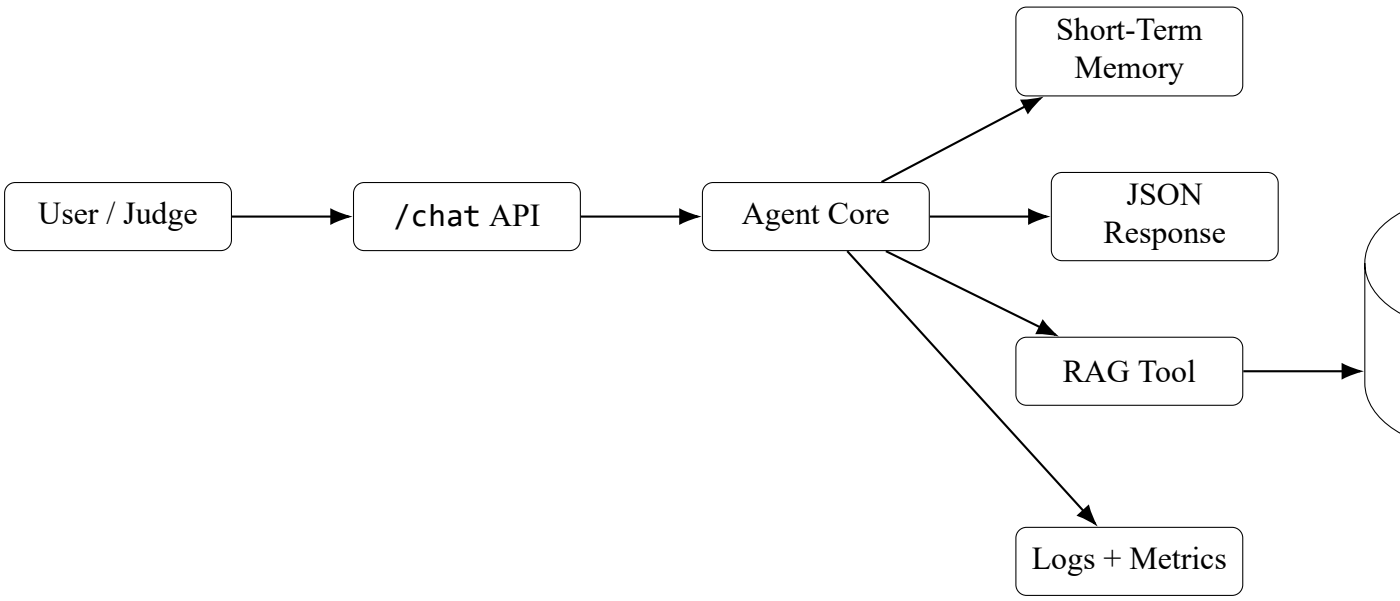


Figure 1: Phase 2 Agentic RAG architecture.

4 Main Components

4.1 HTTP Server

The HTTP server is implemented with Python's standard `http.server`. It provides four routes:

- `GET /`: browser-based manual test page.
- `GET /health`: health check endpoint.
- `GET /metrics`: runtime metrics endpoint.
- `POST /chat`: required judge endpoint.

Main file:

```
phase2/src/agentic_rag/server.py
```

4.2 Agent Core

The agent receives a parsed chat request and decides which behavior is needed. The implemented intent categories are:

- product search,
- product feature question,
- seller or price question,
- multi-turn seller hunt,
- product comparison,
- image-to-base request,

- guardrail or out-of-scope response.

Main file:

```
phase2/src/agentic_rag/agent.py
```

4.3 Phase 1 RAG Tool

The RAG tool loads real Phase 1 artifacts:

```
phase1/torob_rag/artifacts/clean/unified_base_products.parquet
phase1/torob_rag/artifacts/clean/members_clean.parquet
phase1/torob_rag/artifacts/embeddings/clip_v1/image/image_index.parquet
```

It provides three main operations:

- `search_products(query, top_k)`: returns base products.
- `get_members(base_random_key, intent)`: returns seller-level member products.
- `image_to_base(image_base64)`: returns one base product key from the Phase 1 image index.

Main file:

```
phase2/src/agentic_rag/rag.py
```

4.4 Memory

The short-term memory is keyed by `chat_id`. This is important because the judge may send several consecutive requests with the same conversation id.

The memory stores:

- recent dialogue messages,
- extracted city,
- extracted budget,
- extracted style or material,
- extracted product intent.

Runtime memory is saved in:

```
phase2/runtime/memory.json
```

Main file:

```
phase2/src/agentic_rag/memory.py
```

4.5 Monitoring

The implementation includes lightweight local monitoring. It records:

- short-term and semantic memory writes,
- memory reads,
- RAG tool calls,
- agent responses,
- HTTP requests,
- errors.

Generated files:

```
phase2/logs/events.jsonl
phase2/logs/metrics.json
```

Metrics include:

- Mean Time To Answer,
- Tokens Per Message,
- Memory Hit Rate.

5 API Contract

5.1 Endpoint

```
POST /chat
Content-Type: application/json
```

5.2 Request Format

```
{
  "chat_id": "test-1",
  "messages": [
    {
      "type": "text",
      "content": "[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] 1375
    [illegible] [illegible]"
    }
  ]
}
```

5.3 Response Format

```
{
  "message": "[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] ... [illegible] 209000 [illegible].",
  "base_random_keys": ["hhtgaa"],
  "member_random_keys": ["pjwyvz"]
}
```

6 How Each Required Scenario Is Handled

6.1 Scenario 1: Question About a Product Feature

If the user asks about a feature such as material, capacity, width, color, or dimensions, the agent:

1. searches for the product in Phase 1 data,
2. reads the `extra_features` field,
3. extracts the requested attribute if available,
4. returns the answer and the related `base_random_key`.

Relevant code:

```
Agent._feature_question()
Phase1RAGTool.answer_feature()
```

6.2 Scenario 2: Question About Sellers or Prices

If the user asks for the cheapest seller or a seller recommendation, the agent:

1. maps the query to one base product,
2. fetches member products with the same `base_random_key`,
3. sorts by price for cheapest queries,
4. returns one or more `member_random_keys`.

Relevant code:

```
Agent._seller_question()
Phase1RAGTool.get_members()
```

6.3 Scenario 3: Multi-Turn Seller Hunt

For ambiguous seller-hunt requests, the agent first asks for more details. In later requests with the same `chat_id`, it uses memory slots such as budget, city, and style.

Relevant code:

```
Agent._seller_hunt()
MemoryStore.update_slots()
```

6.4 Scenario 4: Comparing Products

For comparison questions, the agent searches each mentioned product and selects the product that best matches the stated use case.

Relevant code:

```
Agent._compare()
Agent._choose_product()
```

6.5 Scenario 5: Image to Base

For image requests, the system accepts a Base64 image message and returns one base product key from the Phase 1 image index. This keeps the required API behavior available without adding a heavy VLM dependency.

Relevant code:

```
Agent._image_response()
Phase1RAGTool.image_to_base()
```

7 Guardrails

The agent refuses requests that try to reveal system prompts or hidden context. If the question is outside the product retrieval domain, the response says that the system is not confident because the question is outside the Phase 1 product data.

Relevant code:

```
Agent._is_injection()
Agent.chat()
```

8 How to Run

8.1 Install Requirements

From the project root:

```
cd "<PROJECT_ROOT>\phase2"
pip install -r .\requirements.txt
```

8.2 Start the Server

```
$env:PHASE1_ARTIFACTS_DIR = "..\phase1\torob_rag\artifacts"
python .\run_server.py --host 127.0.0.1 --port 8000
```

Expected output:

```
Phase 2 Agentic RAG is running at http://127.0.0.1:8000
Use POST /chat. Press Ctrl+C to stop.
```

8.3 Browser Test Page

Open:

```
http://127.0.0.1:8000/
```

The page provides a small form that sends the message to `/chat`.

8.4 Health Check

```
http://127.0.0.1:8000/health
```

Expected result:

```
{"status": "ok"}
```

9 How to Test

9.1 Built-in Tests

```
$env:PHASE1_ARTIFACTS_DIR = "..\phase1\torob_rag\artifacts"
python .\tests\smoke_test.py
python .\tests\http_smoke_test.py
```

9.2 Presentation Demo

The presentation demo starts a temporary `/chat` server, sends the main evaluation scenarios as real HTTP requests, validates the expected response fields, prints each request/response pair, and stops the server automatically.

```
python .\tests\presentation_demo.py
```

9.3 Manual PowerShell Test

```
$body = @{
  chat_id = "test-1"
  messages = @(
    @{
      type = "text"
      content = ???????" ????? ????? ???? ???????? ??????? ? ???????? ?? 1375
   ????? ?????"
    }
  )
} | ConvertTo-Json -Depth 5

Invoke-RestMethod `
  -Method Post `
  -Uri "http://127.0.0.1:8000/chat" `
  -ContentType "application/json; charset=utf-8" `
  -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

10 Example Outputs

10.1 Seller Price Example

Input:

```
[illegible]
 [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] 1375 [illegible] [illegible]
```

Output:

```
{
  "message": "[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]« [illegible]-
   [illegible] [illegible] [illegible] [illegible] [illegible] 1375» [illegible] 209000 [illegible].",
  "base_random_keys": ["hhtgaa"],
  "member_random_keys": ["pjwyvz"]
}
```

11 Limitations

- The text retrieval is a simple lexical search over the Phase 1 cleaned product table. This keeps the service easy to run on the current machine.
- The image scenario returns a base key from the existing Phase 1 image index but does not run a heavy CLIP/VLM model at request time.
- No external LLM API key is required. The response generation is deterministic and based on retrieved evidence.
- The first startup may take some time because the real Phase 1 parquet files are loaded.

12 Conclusion

The Phase 2 implementation satisfies the core project goals with a simple and runnable design. It wraps the Phase 1 RAG artifacts as a tool, provides the required `/chat` endpoint, maintains short-term memory by `chat_id`, returns base and member random keys in the required format, and includes lightweight monitoring and tests.